What the thesis concludes, and what it leaves behind

The evidence divides by what is being measured before it divides by resource

Safety: a difference in kind

Safe Rust rejects all 7 injected memory-safety and data-race classes at compile time. The 7 boundary classes, from a deadlock to a forgotten lock guard, compile and misbehave exactly as they do in C++.

Rust eliminates data races, not race conditions, and aliasing errors, not lock-ordering errors.

Migration: the call tax is bounded

An unmodified Rust core can be driven from C++ across a thin seam that holds every unsafe site. Any call tax is bounded at 10.91%.

API design and training time were not measured.

Performance: two different answers

The round-trip is only 0.250% language work. The median gap is small. On a quiet host the far tail sits lower in Rust.

Timed apart, the core call is 1.6× faster in Rust. Past saturation, service time stays unknown.

What this adds to the decision is an instrument, not a verdict

- A frozen specification
- A parity-tested pair
- A reproducible framework
- A catalogue of what the guarantee misses

Where it goes next

a same-host repeat of in-run and deferred fill, so the recorder can be separated from the machine.

- Production traces
- Further subsystems
- Call-site attribution
- Extend the catalogue
- Async runtimes
- Isolate the heaps

Where the service still meets the sent rate, the choice is which defects to make impossible to write.